



RAG Chatbot System Architecture

High-Level System Design for Retrieval-Augmented Generation Course Assistant

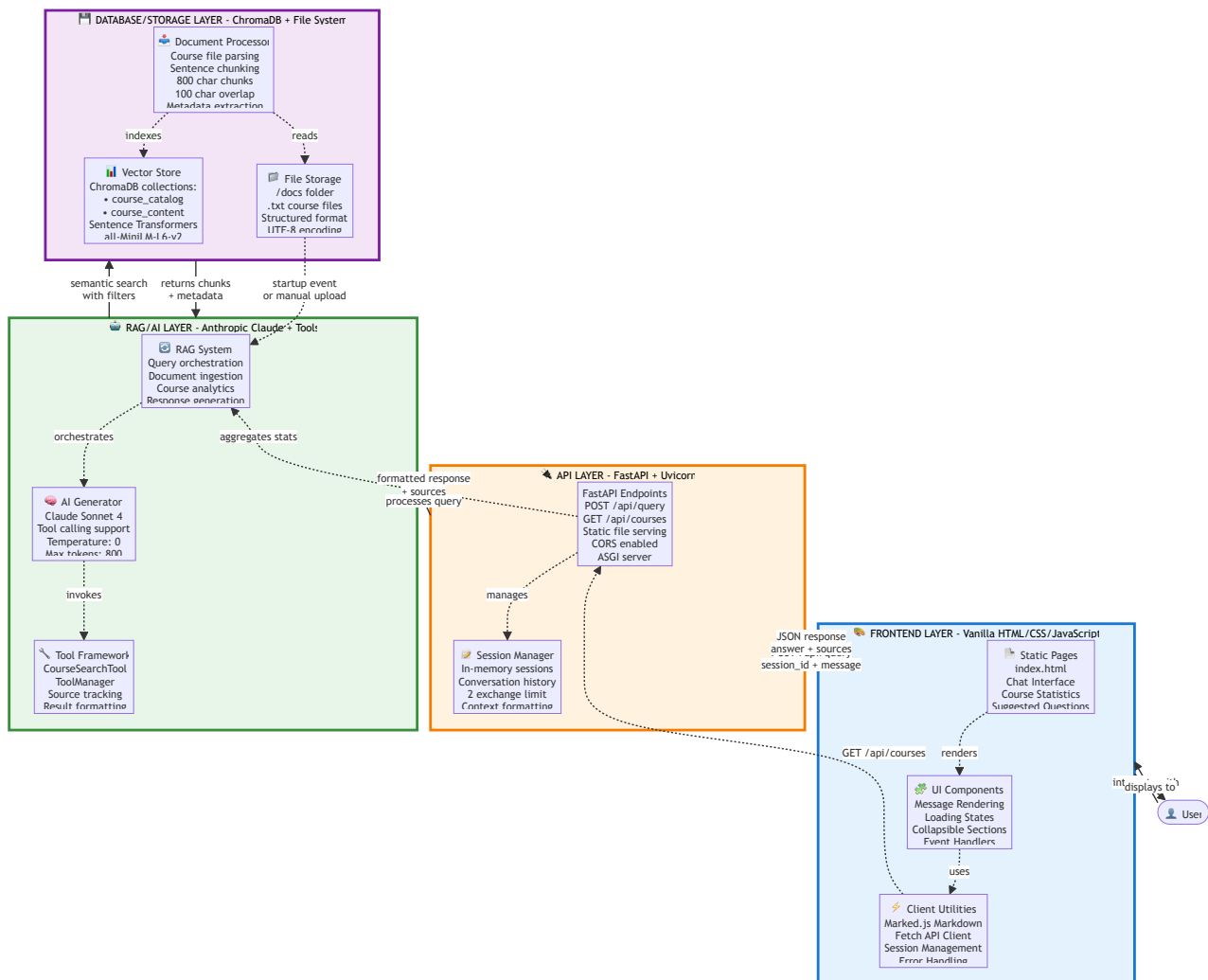
Architecture: Monolithic Full-Stack RAG

Backend: FastAPI + Python 3.13

AI: Anthropic Claude Sonnet 4

Vector DB: ChromaDB

Frontend: Vanilla HTML/CSS/JS





Architecture Components Overview



Frontend Layer

- ▶ **Technology:** Vanilla HTML5/CSS3/JavaScript
- ▶ **Pages:** Single-page chat interface
- ▶ **Components:** Message rendering, loading states
- ▶ **Libraries:** Marked.js for Markdown
- ▶ **State:** Session-based conversation tracking



API Layer

- ▶ **Framework:** FastAPI with Uvicorn ASGI
- ▶ **Endpoints:** /api/query, /api/courses
- ▶ **Sessions:** In-memory with 2 exchange limit
- ▶ **CORS:** Enabled for development
- ▶ **Serving:** Static files + API unified



RAG/AI Layer

- ▶ **AI Model:** Anthropic Claude Sonnet 4
- ▶ **RAG Core:** Query orchestration & ingestion
- ▶ **Tools:** CourseSearchTool with semantic search
- ▶ **Config:** Temperature 0, max 800 tokens
- ▶ **Features:** Tool calling, source tracking



Database/Storage Layer

- ▶ **Vector DB:** ChromaDB (persistent)
- ▶ **Collections:** course_catalog, course_content
- ▶ **Embeddings:** Sentence Transformers (all-MiniLM-L6-v2)

- ▶ **Chunking:** 800 chars with 100 overlap
- ▶ **Files:** Structured .txt course documents

Design Pattern: Layered Architecture with RAG (Retrieval-Augmented Generation) |
Generated: 2025-11-09 | [Built with Claude Code](#)